

Slash *Commands*

Os comandos que você digita **dentro** da sessão do Codex, começando com `/`: configurar o projeto, planejar, revisar, gerenciar o contexto, conectar ferramentas e personalizar a interface, sem sair do terminal.

`/init`

`/plan · /goal`

`/review · /diff`

`/compact`

`/import`

Como funcionam os *slash commands*

Na aula anterior vimos os **comandos do terminal** (`codex exec` , `codex resume` ...). Os slash commands são digitados **no composer, dentro da sessão**, e agem sobre ela sem precisar reiniciar.

1

Digite `/`

Abre o menu com todos os comandos disponíveis.

2

Filtre digitando

`/rev` já mostra `/review`. O menu completa o nome para você.

3

Pressione `Enter`

Alguns abrem um seletor (`/model`), outros agem na hora (`/diff`).

4

Com o Codex trabalhando

Digite o comando e aperte `Tab`: ele entra na fila e roda na próxima rodada.

VÁRIOS COMANDOS ACEITAM TEXTO NA MESMA LINHA

```
/plan Proponha a migração do banco para Postgres
/goal Terminar a migração com os testes passando
/side Esse plano tem algum risco óbvio?
/mention src/lib/api.ts
/clear preparação da release
```

Alguns comandos ficam indisponíveis em certos momentos. Por exemplo, `/plan` não roda com o Codex trabalhando, e `/side` não funciona dentro de outro side chat nem durante um review.

Preparando a *sessão*

Os primeiros comandos de um projeto novo: dar contexto ao Codex e ajustar modelo e permissões.

`/INIT`

Cria o **AGENTS.md**

Gera um `AGENTS.md` inicial no diretório atual, com as instruções persistentes do projeto (comandos de teste, convenções). É o equivalente ao `CLAUDE.md`.

`/STATUS`

Onde estou?

Mostra o modelo ativo, a política de aprovação, as pastas com permissão de escrita e quanto contexto ainda resta.

`/PERMISSIONS`

O que ele pode fazer

Ajusta no meio da sessão o que o Codex faz sem perguntar, por exemplo alternando entre Auto e Read Only.

`/MODEL`

Modelo e raciocínio

Troca o modelo (ex.: `gpt-6-luna`, `gpt-6-sol`) e, quando disponível, o esforço de raciocínio.

`/FAST`

Modo rápido

Liga ou desliga o tier *Fast* do modelo, quando o catálogo oferece. Se o modelo não tem esse tier, o comando nem aparece.

`/PERSONALITY`

Estilo das respostas

Deixa o Codex mais conciso, mais explicativo ou mais colaborativo, sem mudar as suas instruções.

Ritual de início sugerido: `/init` (só na primeira vez) → revisar o `AGENTS.md` gerado → `/status` para conferir modelo e permissões → começar a tarefa.

Planejar e *guiar* o trabalho

Em tarefas maiores, vale alinhar o plano antes de o Codex sair editando arquivos.

`/PLAN`

Coloca o chat em **modo de planejamento**: o Codex propõe um plano de execução antes de implementar. Aceita o pedido na mesma linha e também imagens coladas.

`/GOAL`

Define um **objetivo persistente** que o Codex acompanha durante uma tarefa longa. Sem argumento, mostra o objetivo atual, que também pode ser editado, pausado, retomado ou limpo.

`/SIDE` · `/BTW`

Abre um **chat lateral temporário** para uma pergunta pontual, sem poluir a conversa principal. O status do chat principal continua visível.

`# 1. alinhar o plano`

`/plan` Adicionar autenticação por e-mail no app

`# 2. fixar o critério de pronto`

`/goal` Login funcionando e testes passando

`# 3. apontar os arquivos certos`

`/mention` src/auth/

`/mention` tests/test_auth.py

`# 4. tirar uma dúvida sem desviar a tarefa`

`/side` Qual a diferença entre bcrypt e argon2?

`/mention` faz o mesmo que digitar `@` no composer: anexa um arquivo ou pasta ao chat. Na extensão da IDE, `/ide` inclui os arquivos abertos e a seleção.

Revisar antes de *commitar*

Terminada a tarefa, estes comandos mostram exatamente o que mudou e dão uma segunda opinião sobre o código.

`/DIFF`

O que mudou

Mostra o diff do Git: mudanças staged, unstaged e **arquivos novos ainda não rastreados**.

`/REVIEW`

Segunda opinião

Revisa a working tree com foco em mudanças de comportamento e testes faltando.

`/COPY`

Copiar resposta

Copia a última resposta ou plano concluído. O atalho é

`Ctrl` + `0`.

`/RAW`

Rolagem crua

Liga o modo de rolagem sem formatação, para selecionar e copiar saídas longas.

```
# fluxo de revisão recomendado
/diff                # 1. veja as mudanças
/review              # 2. peça a revisão
# 3. peça as correções no próprio chat
Corrija os dois problemas apontados e rode os testes
/diff                # 4. confira de novo antes do commit
```

O `/review` usa o modelo da sessão, a menos que você defina `review_model` no `config.toml`. Assim dá para revisar com um modelo diferente do que escreveu o código. Fora da sessão, o equivalente é o `codex review` (aula 1).

Contexto e sessões

Uma conversa longa enche a janela de contexto. Saber quando compactar, recomeçar ou bifurcar mantém o Codex preciso.

COMANDO	EFEITO	QUANDO USAR
<code>/compact</code>	Troca os turnos antigos por um resumo e libera contexto	Depois de uma rodada longa, na mesma tarefa
<code>/new</code>	Chat novo na mesma sessão da CLI	Tarefa nova, mesmo repositório
<code>/clear</code>	Limpa o terminal e o contexto, e começa do zero	Recomeço total (aceita um nome)
<code>/fork</code>	Bifurca o chat atual num novo	Testar outra abordagem
<code>/resume</code>	Retoma um chat salvo	Voltar a uma tarefa anterior

COMANDO	EFEITO
<code>/rename</code>	Dá um nome reconhecível à sessão
<code>/archive</code>	Arquiva a sessão e sai (a transcrição continua salva)
<code>/delete</code>	Apaga a sessão de vez e sai
<code>/app</code>	Continua a sessão no app desktop (macOS/Windows)
<code>/exit</code> · <code>/quit</code>	Sai da CLI (atalho <code>Ctrl</code> + <code>C</code>)

Salve ou commite o trabalho importante antes de `/quit` , e prefira `/archive` a `/delete` quando houver dúvida.

Ferramentas, skills e *agentes*

Comandos para ver e usar o que amplia o Codex: servidores MCP, skills, plugins, apps, hooks e subagentes.

/MCP

Lista as ferramentas MCP disponíveis na sessão. `/mcp verbose` mostra o diagnóstico de cada servidor.

/SKILLS

Navega pelas skills locais e insere a escolhida, para que o próximo pedido siga as instruções dela.

/PLUGINS

Mostra os plugins instalados e os disponíveis: ferramentas, instalação e ativação.

/APPS

Navega pelos apps (conectores) e insere no prompt como `$app-slug`.

/HOOKS

Mostra e gerencia os hooks de ciclo de vida: confiar em hooks novos ou alterados, ou desativar os não gerenciados.

/AGENT · /SUBAGENTS

Alterna entre as threads dos subagentes criados, para inspecionar ou continuar o trabalho de cada um.

/PS · /STOP

O `/ps` mostra os terminais em segundo plano (ex.: servidor de dev) e a saída recente de cada um. O `/stop` encerra todos eles.

Se o revisor automático recusar uma ação, `/approve` libera **uma** nova tentativa daquela ação.

Personalização e *diagnóstico*

Para deixar a interface do seu jeito e investigar quando algo não se comporta como esperado.

PERSONALIZAR	O QUE FAZ	DIAGNOSTICAR	O QUE FAZ
<code>/statusline</code>	Escolhe e ordena os itens do rodapé: modelo, contexto, limites, Git, tokens	<code>/usage</code>	Uso de tokens da conta (diário, semanal, acumulado)
<code>/title</code>	Define o título da janela ou aba do terminal	<code>/debug-config</code>	Mostra as camadas de configuração e os requisitos
<code>/theme</code>	Tema de destaque de sintaxe	<code>/experimental</code>	Liga recursos experimentais
<code>/keymap</code>	Remapeia os atalhos da interface	<code>/memories</code>	Controla o uso e a geração de memórias
<code>/vim</code>	Liga ou desliga o modo Vim no composer	<code>/feedback</code>	Envia logs aos mantenedores
<code>/pets</code>	Escolhe ou esconde o "pet" do terminal	<code>/logout</code>	Sai da conta (máquinas compartilhadas)

Só no Windows: `/setup-default-sandbox` configura o sandbox elevado, e `/sandbox-add-read-dir` libera a leitura de uma pasta extra.

Vindo do *Claude Code*

Muitos comandos têm o mesmo nome e o mesmo papel nas duas ferramentas. E o Codex consegue importar a sua configuração do Claude Code.

OBJETIVO	CLAUDE CODE	CODEX
Instruções do projeto	<code>/init</code> → CLAUDE.md	<code>/init</code> → AGENTS.md
Liberar contexto	<code>/compact</code>	<code>/compact</code>
Recomeçar	<code>/clear</code>	<code>/clear</code> · <code>/new</code>
Trocar modelo	<code>/model</code>	<code>/model</code>
Servidores MCP	<code>/mcp</code>	<code>/mcp</code>
Retomar sessão	<code>/resume</code>	<code>/resume</code>
Permissões	<code>/permissions</code>	<code>/permissions</code>
Estado da sessão	<code>/status</code>	<code>/status</code>

`/IMPORT`

Abre um seletor para migrar do **Claude Code** (ou do Cursor) a **configuração, os arquivos de projeto e os chats recentes** (até 50 chats dos últimos 30 dias).

`/import`

`#` → escolha "Claude Code"
`#` → marque configuração, projetos e/ou chats

O `/import` só roda numa sessão local da TUI e fica indisponível com uma tarefa em andamento ou em sessões remotas.

Criando os seus *próprios comandos*

O Codex ainda aceita **custom prompts** (arquivos Markdown que viram `/prompts:nome`), mas a documentação os marca como **descontinuados**. O caminho recomendado agora são as **skills**.

LEGADO: `~/CODEX/PROMPTS/DRAFTPR.MD`

```
---
description: Cria branch, commit e PR em rascunho
argument-hint: [FILES=<paths>] [PR_TITLE=<título>]
---
Crie um branch dev/<nome> para este trabalho.
Se houver arquivos, faça stage deles: $FILES.
Faça o commit com uma mensagem clara.
Abra um PR em rascunho. Use $PR_TITLE, se houver.
```

Uso: `/prompts:draftpr FILES=src/app.ts PR_TITLE="Login"`. Aceita `$1` ... `$9`, `$ARGUMENTS` e placeholders nomeados.

	CUSTOM PROMPTS	SKILLS
Situação	Descontinuados	Recomendadas
Onde ficam	Só em <code>~/codex/prompts/</code>	Podem ir no repositório
Compartilhar	Não vão pelo Git	Sim, com a equipe
Invocação	Só explícita	Explícita (<code>/skills</code>) ou automática

É o mesmo conceito das **Agent Skills** que você viu no módulo 17 do curso: instruções reutilizáveis que o agente usa quando a tarefa pede.

Cola: os slash commands *por categoria*

SESSÃO E MODELO	
<code>/init</code>	Gera AGENTS.md
<code>/status</code>	Config e contexto
<code>/model</code>	Modelo e raciocínio
<code>/fast</code>	Tier rápido
<code>/personality</code>	Estilo de resposta
<code>/permissions</code>	Aprovações
PLANEJAR	
<code>/plan</code>	Modo plano
<code>/goal</code>	Objetivo persistente
<code>/side</code> · <code>/btw</code>	Chat lateral
<code>/mention</code>	Anexa arquivo
<code>/ide</code>	Contexto da IDE

REVISAR	
<code>/diff</code>	Diff do Git
<code>/review</code>	Code review
<code>/copy</code>	Cópia resposta
<code>/raw</code>	Rolagem crua
CONTEXTO E SESSÕES	
<code>/compact</code>	Resume o chat
<code>/new</code> · <code>/clear</code>	Recomeçar
<code>/fork</code> · <code>/resume</code>	Bifurcar · retomar
<code>/rename</code>	Nomear sessão
<code>/archive</code> · <code>/delete</code>	Arquivar · apagar
<code>/app</code> · <code>/quit</code>	App desktop · sair

EXTENSÕES	
<code>/mcp</code> · <code>/skills</code>	MCP · skills
<code>/plugins</code> · <code>/apps</code>	Plugins · apps
<code>/hooks</code>	Hooks
<code>/agent</code>	Subagentes
<code>/ps</code> · <code>/stop</code>	Terminais em background
<code>/approve</code>	Nova tentativa
<code>/import</code>	Do Claude Code
AJUSTES	
<code>/statusline</code> · <code>/theme</code>	Visual
<code>/usage</code> · <code>/debug-config</code>	Diagnóstico

Teste *rápido*

1. A conversa ficou longa, mas você quer continuar na mesma tarefa sem perder os pontos importantes. Qual comando usar?

A) /clear

B) /new

C) /compact

D) /delete

C. O `/compact` troca os turnos antigos por um resumo e libera contexto. O `/clear` e o `/new` começam do zero.

2. O Codex está no meio de uma rodada e você quer rodar `/diff` assim que ela terminar. O que fazer?

A) Esperar e digitar depois

B) Digitar `/diff` e apertar Tab

C) Apertar Ctrl+C

D) Não é possível

B. Com o Codex trabalhando, `Tab` coloca o slash command na fila para a próxima rodada.

3. Qual comando mostra também os arquivos novos que o Git ainda não rastreia?

A) /review

B) /diff

C) /status

D) /copy

B. O `/diff` mostra as mudanças staged e unstaged e os arquivos ainda não rastreados.

4. Qual é hoje o jeito recomendado de criar instruções reutilizáveis e compartilháveis com a equipe?

A) Custom prompts em `~/.codex/prompts`

B) Skills

C) /goal

D) /personality

B. Os custom prompts estão descontinuados e ficam só na sua máquina. As skills podem ir no repositório e ser usadas de forma automática.

○ RECAPITULANDO

Controle a sessão *sem sair dela*

Com `/init` e `/status` você prepara o projeto. Com `/plan`, `/goal` e `/mention` guia o trabalho, e com `/diff` e `/review` revisa antes do commit. Para o contexto e as sessões há `/compact`, `/new` e `/fork`, para estender o Codex há `/mcp` e `/skills`, e para trazer o Claude Code há `/import`. Na dúvida, digite `/` e explore.

`/init`

`/plan · /goal`

`/diff · /review`

`/compact`

`/skills`

`/import`

Próxima aula → Criando uma landing page

Fontes: learn.chatgpt.com/docs (developer-commands?surface=cli · slash commands in Codex CLI · custom prompts · build-skills · import). A lista de comandos varia com a versão e com os recursos habilitados; digite `/` na sua sessão para ver os disponíveis.